CENTRO UNIVERSITÁRIO DE BRASÍLIA – UniCEUB
CURSO DE ENGENHARIA DE SOFTWARE
DISCIPLINA: BOOTCAMP I

BRENO LOPES SOUZA

IMPLANTAÇÃO DO PORTFOLIOHUB + IA GEMINI

BRASÍLIA – DF
2026

BRENO LOPES SOUZA

IMPLANTAÇÃO DO PORTFOLIOHUB + IA GEMINI

Trabalho acadêmico apresentado à disciplina de Bootcamp I, do curso de Engenharia de Software do Centro Universitário de Brasília – UniCEUB, como parte dos requisitos da entrega final.

Professor: Marcelo Carboni.

BRASÍLIA – DF
2026

RESUMO

Este relatório sintetiza o planejamento e a implantação proposta do PortfolioHUB, portfólio acadêmico estático desenvolvido com HTML, CSS e JavaScript. O projeto organiza informações profissionais e projetos públicos, consulta repositórios por meio da API do GitHub e prevê publicação com GitHub Pages. São detalhados o plano de execução, a estrutura técnica dos arquivos, a gestão de usuários e a política de segurança, as práticas de versionamento e colaboração e os testes de validação. O Google Gemini é apresentado como ferramenta de apoio, com prompts preparados para análise crítica e documentação das respostas reais. A integração com serviços do Google ocorre por links oficiais, sem acesso a contas privadas ou armazenamento de credenciais no navegador. Distingue-se a implementação local das etapas que exigem comprovação no ambiente do estudante, como a URL publicada, os registros de execução e o vídeo de apresentação.

Palavras-chave: PortfolioHUB. GitHub. Git. Google Gemini. Segurança da informação. Implantação.

SUMÁRIO

A. CONTEXTUALIZAÇÃO E REQUISITOS 5
A.1 – Contextualização da atividade e objetivos 5
A.2 – Levantamento dos requisitos e rastreabilidade 5
1. PLANEJAMENTO DA IMPLANTAÇÃO 7
1.1 – Plano de implantação detalhado 7
1.2 – Arquitetura do PortfolioHUB 7
1.3 – Cronograma, recursos e critérios de sucesso 8
1.4 – Gestão de riscos e qualidade 8
2. CONFIGURAÇÃO INICIAL E INTEGRAÇÃO COM GITHUB 10
2.1 – Configuração inicial do GitHub 10
2.2 – Organização dos arquivos e do código 10
2.3 – Consulta de repositórios pela API do GitHub 11
2.4 – Integração com ferramentas Google 12
2.5 – Publicação com GitHub Pages 12
3. GESTÃO DE USUÁRIOS E SEGURANÇA 14
3.1 – Gestão de usuários e responsabilidades 14
3.2 – Política de segurança da informação 14
3.3 – Privacidade, LGPD e informações públicas 15
3.4 – Tratamento de incidentes e recuperação 15
4. COMPARTILHAMENTO E CONTROLE DE ACESSO 17
4.1 – Fundamentos de versionamento e commits 17
4.2 – Branches e pull requests 17
4.3 – Colaboração e controle de acesso 18
4.4 – Documentação do repositório 18
5. FINALIZAÇÃO DA INTEGRAÇÃO E TESTES 19
5.1 – Plano de testes funcionais 19
5.2 – Testes de usabilidade e acessibilidade 19
5.3 – Testes de segurança e confiabilidade 20
5.4 – Validação automatizada e integração contínua 20
5.5 – Critérios de lançamento e manutenção 21
B. ROTEIRO DE PROMPTS PARA O GOOGLE GEMINI 22
B.1 – Metodologia e registro de interações 22
B.2 – Prompts de planejamento e implantação 22
Prompt 1 – Contexto acadêmico 22
Prompt 2 – Planejamento da implantação 22
Prompt 3 – Arquitetura e GitHub 23
Prompt 4 – Segurança e dados pessoais 23
Prompt 5 – Colaboração 23
Prompt 6 – Google Workspace 23
B.3 – Prompts de testes e aperfeiçoamento 23
Prompt 7 – Testes funcionais 23
Prompt 8 – Acessibilidade 24
Prompt 9 – Implantação GitHub Pages 24
Prompt 10 – Revisão crítica 24
Prompt 11 – Apresentação de 5 minutos 24
Prompt 12 – Melhorias futuras 24
REFERÊNCIAS 25

A. CONTEXTUALIZAÇÃO E REQUISITOS

A.1 – Contextualização da atividade e objetivos

A implantação do PortfolioHUB demonstra o processo de transformar uma coleção de projetos e documentos acadêmicos em uma presença digital organizada, acessível e versionada. O resultado almejado é uma vitrine profissional progressivamente atualizada, com origem de código auditável e separação entre conteúdo público e conteúdo restrito.

- Interpretar as seis frentes do enunciado como requisitos verificáveis e organizá-las em entregáveis: código, documentação, testes e apresentação.
- Escolher GitHub Pages para hospedagem estática e GitHub para controle de alterações, sem necessidade de credenciais expostas no navegador.
- Identificar as seções do site: apresentação, objetivos, projetos públicos e acesso a ferramentas externas.
- Organizar a distinção entre conteúdo demonstrativo e informações originadas da API pública do GitHub.

Validação prevista: O plano relaciona cada exigência a um arquivo ou evidência; O site pode ser aberto localmente e seus links possuem destinos definidos.

Limitação e risco: Risco principal: confundir um protótipo estático com uma plataforma de autenticação e permissões completas. Para mitigar isso, descrevem-se explicitamente as fronteiras da solução e a configuração real delegada ao GitHub.

A.2 – Levantamento dos requisitos e rastreabilidade

O projeto usa os requisitos do PDF final como base. A rastreabilidade associa os seis itens de avaliação a evidências técnicas, permitindo que a banca verifique a solução sem depender exclusivamente de afirmações narrativas.

Objetivo: Definir uma relação objetiva entre necessidade, artefato produzido, ação de validação e evidência a anexar.

- R1 Planejamento: documento de implantação com objetivos, fases, cronograma e riscos.
- R2 GitHub: estrutura do repositório com docs/, README.md, .gitignore e workflow.
- R3 Segurança: política documentada, decisão de não incorporar segredos e matriz de permissões.
- R4 Compartilhamento: prática de commits semânticos, branches, pull requests e permissões de colaboradores.

- R5 Testes: checklist técnico com resultados esperados e campos para execução.
- R6 Apresentação: roteiro de cinco minutos, capturas e links de evidência.

Validação prevista: Cada requisito tem uma seção no relatório; Os caminhos citados existem no projeto entregue, salvo itens assinalados como evidência externa.

Limitação e risco: O conteúdo original do exemplo acadêmico de 2024 não deve ser tratado como demonstração de atividades executadas por outro estudante. Portanto, a estrutura serve de inspiração, não de fonte de evidências operacionais.

1. PLANEJAMENTO DA IMPLANTAÇÃO

1.1 – Plano de implantação detalhado

O ciclo de implantação foi dividido em fases pequenas, com entregáveis independentes e critérios de aceite observáveis. Essa abordagem facilita o controle de qualidade e reduz a probabilidade de publicar material incompleto.

- Fase 1 – Diagnóstico: levantar objetivos, mapear o perfil acadêmico e delimitar o conteúdo público.
- Fase 2 – Preparação: estruturar pastas de código, documentos e automatização.
- Fase 3 – Interface: criar HTML semântico, CSS responsivo e JavaScript de consulta pública.
- Fase 4 – Segurança: examinar links, tokens, permissões e documentação do repositório.
- Fase 5 – Validação: executar checklist e revisar comportamento sob falha de API.
- Fase 6 – Implantação: realizar push, configurar Pages e testar o endereço publicado.
- Fase 7 – Entrega: registrar evidências, usar Gemini de modo verificável, gravar vídeo e fechar PDF.

Validação prevista: O responsável e o estado das fases aparecem no registro do projeto; A publicação é considerada concluída somente após abertura real da URL.

Limitação e risco: Falhas de rede e falta de disponibilidade dos serviços externos podem atrasar a validação. Para mitigar, manter a interface demonstrativa e repetir os testes quando a conexão voltar.

1.2 – Arquitetura do PortfolioHUB

A aplicação segue uma arquitetura frontend estática: o navegador carrega os arquivos HTML, CSS e JavaScript; o JavaScript consulta exclusivamente endpoints públicos e apresenta os resultados na interface. O GitHub atua separadamente como ambiente de versionamento e publicação.

- Camada de apresentação – docs/index.html: conteúdo, estrutura semântica, seções e identificadores usados pelo JavaScript.
- Camada visual – docs/style.css: hierarquia, responsividade, adaptação ao viewport e variações de tema.
- Camada de comportamento – docs/app.js: pesquisa, ordenação, alternância de tema, renderização de cartões e comunicação com API.

- Camada de dados públicos – API REST do GitHub para repositórios públicos do usuário BrenoLSouza.
- Camada de operação – repositório Git, documentação em Markdown e workflow de verificação.
- Camada de hospedagem – GitHub Pages após ativação manual no repositório.

Validação prevista: Nenhuma credencial de acesso é inserida no JavaScript; Os cartões usam links públicos e texto inserido de forma segura.

Limitação e risco: A API pública pode impor limites de requisição, alterar formato ou ficar indisponível. A aplicação já possui tratamento de falha para manter uma experiência demonstrativa; os exemplos locais não constituem confirmação de repositórios existentes.

1.3 – Cronograma, recursos e critérios de sucesso

O planejamento emprega uma sequência lógica de tarefas e dependências. A duração de cada fase deve ser adaptada às restrições reais do estudante; a data de entrega do enunciado é 25 de outubro de 2026 às 23h55.

- Planejamento e inventário de projetos: definir quais informações pessoais serão públicas.
- Desenvolvimento do site: estruturar navegação, seções, cartões e responsividade.
- Documentação: produzir README, política de segurança e instruções de Git.
- Revisão funcional: avaliar busca, ordenação, estados vazios e links externos.
- Revisão de segurança: confirmar ausência de tokens, senhas e material sigiloso.
- Publicação: executar Git push, configurar Pages e registrar endereço.
- Apresentação: ensaiar e gravar demonstração orientada pelos critérios de avaliação.

Validação prevista: Critério de sucesso 1: site carrega pelo endereço público com todos os recursos estáticos; Critério de sucesso 2: repositórios públicos aparecem ou a mensagem alternativa é exibida.

Limitação e risco: O principal risco do cronograma é deixar publicação e registro de evidência para a última etapa. Prioriza-se o caminho crítico: site funcional, repositório publicado e documentação comprobatória.

1.4 – Gestão de riscos e qualidade

Os riscos foram classificados pela consequência na entrega e pela capacidade de detectar falhas antes da publicação. O gerenciamento de riscos considera disponibilidade, confidencialidade, integridade, usabilidade e entregabilidade acadêmica.

- Risco: token ou senha no repositório. Controle: não inserir credenciais em frontend e verificar alterações antes do commit.
- Risco: acesso público indevido a documentos. Controle: publicar somente conteúdo revisado e conferir permissões dos links.
- Risco: erro na API GitHub. Controle: tratamento de erros, aviso acessível e conteúdo demonstrativo identificado.
- Risco: quebra em celulares. Controle: testar larguras e interações responsivas.
- Risco: contribuição não revisada. Controle: branch e pull request, quando disponíveis e configurados.
- Risco: ausência de prova de uso do Gemini. Controle: registrar prompts, data e captura real.

Validação prevista: Revisão do código antes de cada atualização pública; Checklist de critérios de segurança aplicado antes do lançamento.

Limitação e risco: Não basta pedir ao Gemini para aprovar a segurança: IA é apoio à análise, não mecanismo técnico de proteção. O estudante deve verificar controles nas interfaces e documentação oficial das plataformas.

2. CONFIGURAÇÃO INICIAL E INTEGRAÇÃO COM GITHUB

2.1 – Configuração inicial do GitHub

O ambiente de trabalho previsto é a conta GitHub BrenoLSouza, com um repositório dedicado ao PortfolioHUB. A criação efetiva, visibilidade, permissões e seleção da branch principal dependem de acesso ao GitHub do estudante.

- Criar um repositório denominado PortfolioHUB na conta BrenoLSouza e definir visibilidade pública apenas quando o conteúdo estiver revisado.
- Clonar o repositório ou inicializar a pasta local com Git, assegurando que somente arquivos pertinentes sejam rastreados.
- Adicionar README.md com descrição, arquitetura, execução local, evidências, riscos e limites da solução.
- Manter o diretório docs/ como raiz publicável do site no GitHub Pages.
- Registrar alterações em commits pequenos, específicos e inteligíveis.
- Abrir as configurações do repositório e confirmar branch principal, colaboradores e disponibilidade do Pages.

Validação prevista: URL do repositório deverá ser registrada após criação real; Histórico de commits deverá mostrar o desenvolvimento.

Limitação e risco: Nunca declarar que o repositório já existe somente porque há referências à URL planejada no código. Confirmar a criação visitando a página pública.

2.2 – Organização dos arquivos e do código

A pasta do projeto foi estruturada para separar site, documentação e verificação automatizada. Essa separação melhora a manutenção do código e ajuda o professor a localizar as evidências.

- docs/index.html: estrutura de navegação, seções informativas, links externos e âncoras do portfólio.
- docs/style.css: tipografia, cores, layouts, cartões e regras para telas pequenas.
- docs/app.js: consumo da API pública, criação de elementos, busca, ordenação, alternância de tema e fallback.
- documentacao/PLANO_DE_IMPLANTACAO.md: resumo operacional com objetivos e entregáveis.

- documentacao/POLITICA_DE_SEGURANCA.md: orientação de proteção e limitação de exposição.
- documentacao/GEMINI_PROMPTS.md: prompts reproduzíveis, sem respostas fictícias.
- documentacao/ROTEIRO_VIDEO_5_MIN.md: apresentação orientada às etapas.
- .github/workflows/validate.yml: automação de validação incluída no pacote.

Validação prevista: Abrir cada arquivo no editor e validar os caminhos locais; Inspecionar HTML/CSS/JS separadamente.

Limitação e risco: Um projeto organizado não implica qualidade sem execução de testes e revisão dos resultados. A organização é condição de manutenção, mas a validação exige execução verificável.

2.3 – Consulta de repositórios pela API do GitHub

A implementação utiliza requisições GET à API pública de repositórios do usuário. A listagem é tratada como dado externo não confiável: nomes, descrições e tecnologias são apresentados como texto e não injetados como HTML.

Objetivo: Exibir projetos públicos dinamicamente, evitando atualização manual de todos os cartões.

- Definir o identificador público BrenoLSouza como referência da consulta.
- Consultar /users/BrenoLSouza/repos com parâmetros per_page e sort.
- Checar a resposta HTTP; em caso de falha, apresentar mensagem e itens locais demonstrativos.
- Converter os campos de cada objeto em modelo simples: nome, descrição, URL, linguagem, estrelas e data.
- Filtrar os resultados conforme texto digitado pelo visitante.
- Ordenar por data de atualização, nome ou estrelas conforme seleção do usuário.
- Abrir URLs externas em nova aba com atributo rel apropriado.

Validação prevista: Requisição bem-sucedida mostra repositórios públicos retornados; Falha de rede exibe aviso inteligível ao visitante.

Limitação e risco: Dados públicos não requerem autenticação no cenário proposto. Contudo, a API impõe limites e os resultados podem mudar sem aviso; não se deve prometer disponibilidade contínua.

2.4 – Integração com ferramentas Google

O desafio contempla o Gemini como assistente e a integração do ecossistema Google pode apoiar o fluxo acadêmico. Na versão disponibilizada, a integração com Docs, Drive, Gmail e Agenda ocorre por links externos; não há API privada ou sessão unificada.

- Google Docs: manter o relatório e versões de planejamento com compartilhamento definido pelo proprietário.
- Google Drive: armazenar os documentos e evidências, diferenciando pastas pessoais e materiais públicos.
- Google Agenda: acompanhar atividades, revisões e prazos de entrega.
- Gmail: utilizar comunicação institucional com professor e colegas.
- Google Gemini: revisar decisões de projeto a partir de prompts explicitamente documentados.
- Para cada documento compartilhado, avaliar se o destinatário precisa de visualização ou edição.

Validação prevista: Os links levam aos serviços oficiais; Nenhum iframe presume acesso à caixa de entrada privada.

Limitação e risco: Incorporar interfaces privadas por iframe pode ser bloqueado por políticas de segurança dos serviços; por isso, a versão acadêmica utiliza links diretos, sem alegar uma integração por API.

2.5 – Publicação com GitHub Pages

O GitHub Pages permite hospedar os arquivos estáticos do projeto a partir de uma branch e diretório selecionados no repositório. A publicação precisa ser efetivamente configurada pelo proprietário.

- Abrir o repositório e navegar às configurações de Pages.
- Selecionar publicação por branch e diretório /docs, quando essa opção estiver disponível.
- Salvar e aguardar o serviço disponibilizar o endereço do site.
- Visitar o endereço em janela sem autenticação para verificar acesso externo.
- Confirmar que CSS e JavaScript foram carregados com caminhos relativos corretos.
- Registrar URL final, data de teste e captura da página inicial em execução.

Validação prevista: URL real do site: preencher após ativação; Captura deve mostrar a página renderizada no domínio do Pages.

Limitação e risco: É possível que haja atraso de propagação após configurar Pages; verificar mensagens de build e workflow no repositório antes de concluir que a publicação falhou.

3. GESTÃO DE USUÁRIOS E SEGURANÇA

3.1 – Gestão de usuários e responsabilidades

O modelo atual não mantém banco de usuários nem página de autenticação. A gestão de permissões relacionada ao código e à colaboração ocorre no GitHub, na camada de repositório, não dentro do site público.

- Proprietário: configura repositório, políticas de colaboração, publicações e materiais públicos.
- Colaborador autorizado: propõe alterações de código conforme acesso concedido.
- Contribuidor externo: pode usar fork e pull request em repositório público quando permitido.
- Visitante: visualiza o site e acessa links públicos, sem permissão de editar conteúdo.
- Auditor ou professor: consulta materiais de entrega sem precisar conhecer senhas.

Validação prevista: Matriz de acesso corresponde às capacidades reais da plataforma; Nenhum botão de login fictício é apresentado como autenticação segura.

Limitação e risco: Uma interface de login sem backend e sem sessão verificada não protegeria conteúdos. A solução evita esse recurso por ser desnecessário ao escopo atual.

3.2 – Política de segurança da informação

A política documentada orienta confidencialidade, integridade e disponibilidade. As práticas são proporcionais ao risco de um site estático público e ao uso de serviços externos.

Objetivo: Mitigar vazamentos, alterações indevidas e dependência de acesso não controlado.

- Não armazenar chaves de API privadas, senhas ou tokens no repositório e no JavaScript enviado ao navegador.
- Ativar autenticação multifator na conta GitHub e proteger seus mecanismos de recuperação.
- Aplicar o princípio do menor privilégio ao convidar colaboradores.
- Revisar alterações de terceiros antes de fazer merge na branch utilizada em produção.
- Não publicar documentos pessoais ou dados de terceiros sem necessidade e autorização.
- Manter backups do repositório ou cópia local versionada em mídia de confiança.
- Revisar periodicamente colaboradores, aplicativos autorizados e URLs públicas.

Validação prevista: Código-fonte não contém tokens privados; Permissões de compartilhamento são examinadas antes da publicação.

Limitação e risco: Ferramentas de IA ajudam a identificar medidas, mas não garantem conformidade. Normas como ISO/IEC 27001, NIST CSF e LGPD são referências de princípios; não se declara certificação ou auditoria formal.

3.3 – Privacidade, LGPD e informações públicas

Mesmo um portfólio acadêmico deve limitar exposição desnecessária de informações pessoais. O visitante pode acessar dados do autor, mas isso não implica autorização para publicar dados de colegas ou instituições sem critério.

- Listar dados pessoais realmente necessários: nome, curso, projetos e contatos escolhidos pelo autor.
- Excluir números de documentos, endereços residenciais, dados financeiros e informações confidenciais.
- Verificar direitos de uso das imagens e capturas.
- Publicar links de documentos apenas após conferir suas permissões.
- Oferecer canal de atualização ou remoção de dados quando aplicável.

Validação prevista: Material público foi selecionado conscientemente; Não há coleta silenciosa de dados sensíveis pela aplicação estática.

Limitação e risco: A simples menção à LGPD não prova cumprimento integral da legislação. A análise apresentada é uma aplicação proporcional de seus princípios ao escopo acadêmico.

3.4 – Tratamento de incidentes e recuperação

Falhas de publicação, remoção acidental de arquivos e exposição de dados exigem respostas previsíveis. Os mecanismos básicos de recuperação podem ser implementados por versionamento e revisão de permissões.

- Identificar o incidente: determinar qual arquivo, conta ou conteúdo foi afetado.
- Conter: retirar temporariamente material indevido e revogar links ou credenciais quando necessário.
- Analisar histórico de commits e alterações administrativas para localizar a origem.
- Corrigir o problema e verificar se o conteúdo publicado reflete a versão segura.
- Registrar ocorrido, responsável, data e medidas preventivas.

Validação prevista: Existe um procedimento descrito de detecção, contenção, correção e recuperação; A versão restaurada é submetida ao checklist de qualidade.

Limitação e risco: Remover um segredo de um arquivo não garante que ele desapareça do histórico Git. Caso haja exposição de credenciais, revogá-las e substituí-las imediatamente.

4. COMPARTILHAMENTO E CONTROLE DE ACESSO

4.1 – Fundamentos de versionamento e commits

Git registra alterações locais, enquanto GitHub oferece uma plataforma para compartilhamento e revisão do trabalho. O benefício acadêmico inclui comprovar a evolução do projeto e justificar decisões de implementação.

- Executar git status para revisar alterações preparadas.
- Adicionar somente arquivos intencionais com git add.
- Criar commits com mensagens claras, por exemplo: feat: adiciona consulta de projetos públicos.
- Separar melhorias visuais, documentação e correções em commits específicos.
- Enviar a branch para o repositório remoto com git push.
- Inspecionar histórico no GitHub e conferir os arquivos publicados.

Validação prevista: Histórico mostra alterações distintas e explicáveis; Arquivos locais e remotos correspondem à versão pretendida.

Limitação e risco: Commits gigantes dificultam auditoria e recuperação. Commits pequenos permitem identificar regressões e tornar a defesa acadêmica mais demonstrável.

4.2 – Branches e pull requests

Branches permitem desenvolver melhorias sem modificar imediatamente o ramo usado na publicação. Pull requests documentam a proposta, revisão e decisão de integração.

- Criar branch com nome descritivo, como feature/melhora-cartoes.
- Alterar os arquivos envolvidos e testar localmente.
- Publicar a branch e abrir pull request contra a principal.
- Descrever objetivo, arquivos modificados, testes feitos e capturas relevantes.
- Revisar diferenças de código antes da aprovação.
- Concluir merge quando o resultado estiver validado e registrar eventual rollback.

Validação prevista: Mudanças entram na branch principal somente após revisão prevista; Cada pull request contém descrição e evidências de teste.

Limitação e risco: A exigência de revisão obrigatória depende do tipo de repositório e de suas configurações de proteção. Este relatório não afirma que a regra já está ativa.

4.3 – Colaboração e controle de acesso

O controle de colaboração precisa ser proporcional ao projeto e auditável. O proprietário decide quem pode alterar repositório, configurações e conteúdos públicos.

- Definir proprietário como responsável pela publicação e revisão final.
- Conceder acesso de escrita somente quando estritamente necessário.
- Preferir issues para registrar ideias, dúvidas e bugs.
- Utilizar pull requests para receber contribuições em vez de compartilhar credenciais.
- Retirar acesso quando a colaboração terminar.
- Documentar boas práticas de commits, convenções e critérios de aceite.

Validação prevista: Permissões do repositório são revisadas pelo proprietário; Documentação de contribuição está disponível.

Limitação e risco: A URL pública do site permite leitura, não escrita. A separação é essencial: divulgar o portfólio não equivale a conceder acesso de edição.

4.4 – Documentação do repositório

A documentação deve ser útil a uma pessoa que nunca viu o projeto. Ela apresenta contexto, como executar, arquitetura, segurança, limitações, colaboração e publicação.

- Escrever descrição do PortfolioHUB e objetivos acadêmicos no README.
- Registrar a estrutura de pastas e a responsabilidade de cada arquivo.
- Explicar abertura local de docs/index.html e dependência de conexão para consulta remota.
- Documentar publicação via Pages sem prometer URL ainda não confirmada.
- Manter política de segurança, prompts Gemini, checklist e roteiro de vídeo em arquivos separados.
- Adicionar links das evidências após a conclusão de cada etapa.

Validação prevista: Usuário externo consegue identificar o arquivo inicial; Instruções não dependem de credenciais compartilhadas.

Limitação e risco: Documentação desatualizada pode ser tão prejudicial quanto ausência dela. É importante revisá-la sempre que a arquitetura mudar.

5. FINALIZAÇÃO DA INTEGRAÇÃO E TESTES

5.1 – Plano de testes funcionais

Os testes não devem ser tratados apenas como uma etapa de confirmação visual. É necessário especificar entrada, ação, resultado esperado e evidência do resultado real.

- TF01 – Abrir site: verificar título, menu, seções e carregamento de estilos.
- TF02 – Consulta da API: verificar apresentação dos repositórios públicos ou status de falha.
- TF03 – Busca: inserir termo presente e confirmar filtro dos cartões.
- TF04 – Busca sem resultado: inserir termo inexistente e confirmar mensagem de estado vazio.
- TF05 – Ordenação: alternar opções de nome, data e estrelas; observar mudança coerente.
- TF06 – Tema: alternar o botão e verificar legibilidade dos textos.
- TF07 – Links: abrir GitHub e ferramentas externas; conferir URL e comportamento de nova aba.
- TF08 – Erro de rede: simular indisponibilidade da API e confirmar aviso e fallback.

Validação prevista: Cada teste possui campo a preencher com aprovado, reprovado ou não executado; Capturas podem demonstrar os estados mais importantes.

Limitação e risco: Os resultados esperados são especificações, não provas de execução. Sem capturas ou log de teste não é correto marcar aprovação final.

5.2 – Testes de usabilidade e acessibilidade

O PortfolioHUB deve ser navegável por diferentes tamanhos de tela e formas de interação. A revisão privilegia leitura, hierarquia, navegação por teclado e mensagens compreensíveis.

- Abrir a página em larguras típicas de celular, tablet e desktop.
- Inspecionar cortes de texto, cartões sobrepostos e rolagem horizontal inesperada.
- Usar tecla Tab para navegar por links, botões e filtros.
- Verificar foco visível durante navegação por teclado.
- Conferir nomes compreensíveis em controles e mensagens de estado.
- Avaliar contraste e legibilidade em ambos os temas.
- Executar inspeção com ferramentas de acessibilidade como apoio, sem tratar uma pontuação automatizada como auditoria completa.

Validação prevista: Interface permanece legível em tamanhos variados; Controles essenciais são alcançáveis por teclado.

Limitação e risco: Responsividade visual, por si só, não assegura acessibilidade. A verificação manual por teclado complementa a análise automática.

5.3 – Testes de segurança e confiabilidade

Em um site estático que consome dados públicos, a segurança concentra-se em exposição acidental, conteúdo externo, links, permissões de repositório e recuperação perante falhas.

- Pesquisar por strings de token e senha em arquivos rastreados antes do push.
- Conferir se links externos usam atributos de proteção adequados.
- Validar que descrições vindas da API são inseridas como texto simples.
- Desabilitar rede para avaliar o tratamento do erro.
- Inspecionar se informações de demonstração estão identificadas como exemplos.
- Verificar que o Pages não exige login para conteúdo que se deseja tornar público.
- Revisar colaboradores e permissões reais no GitHub após publicação.

Validação prevista: Não há segredos no código distribuído; Entradas externas não são executadas como HTML.

Limitação e risco: O checklist é uma revisão introdutória, não um teste de invasão. Uma aplicação posterior com backend requer auditoria mais profunda, atualização de dependências e monitoramento.

5.4 – Validação automatizada e integração contínua

O repositório inclui uma definição de workflow em .github/workflows/validate.yml para apoiar a verificação do projeto. A presença do arquivo não comprova que o GitHub Actions tenha sido executado com sucesso na conta do aluno.

Objetivo: Reduzir erros repetitivos no ciclo de atualização.

- Inspecionar o arquivo de workflow e confirmar os gatilhos e comandos definidos.
- Realizar push para um repositório sob controle do estudante.
- Abrir a aba Actions para verificar se a execução foi iniciada.
- Ler o resultado de cada etapa e corrigir eventuais falhas.
- Registrar uma captura da execução bem-sucedida se houver.
- Não substituir testes visuais e testes de publicação por um resultado automático limitado.

Validação prevista: O workflow é reconhecido pelo GitHub após o push; Logs e status da execução são documentados.

Limitação e risco: A diferença entre configuração e execução é central: um arquivo YAML local descreve automação potencial, não prova que a integração contínua já passou.

5.5 – Critérios de lançamento e manutenção

A liberação pública depende da conclusão de verificações mínimas e da aprovação dos materiais que ficam acessíveis a terceiros. Publicar não encerra o ciclo de desenvolvimento.

- Revisar site, documentação, links e ausência de dados sensíveis.
- Confirmar a branch e o diretório configurados no GitHub Pages.
- Testar a URL em navegador não autenticado.
- Anotar eventuais erros conhecidos e limitações no README.
- Arquivar capturas, versão do código e resultados manuais de teste.
- Planejar atualizações ao final de cada período letivo ou conclusão de projeto novo.

Validação prevista: Site público abre sem autenticação de editor; Documentos descrevem exatamente as funcionalidades atuais.

Limitação e risco: Projetos acadêmicos ganham credibilidade quando suas limitações são reconhecidas e acompanhadas de plano de evolução.

B. ROTEIRO DE PROMPTS PARA O GOOGLE GEMINI

B.1 – Metodologia e registro de interações

O exemplo fornecido reserva uma parte extensa às interações com IA. Para preservar essa intenção pedagógica sem atribuir ao Gemini respostas não verificadas, este relatório apresenta prompts completos, objetivos de aprendizagem, critérios de revisão e fichas de registro. As respostas do Gemini devem ser obtidas diretamente na conta do estudante e anexadas como evidência, se exigidas.

Recomenda-se criar uma conversa dedicada ao projeto, apresentar o contexto e compartilhar apenas informações públicas. A análise humana deverá verificar orientações sugeridas, particularmente segurança, permissões e configuração de APIs. Gemini não substitui documentação oficial nem executa automaticamente a configuração das contas.

Registro recomendado para cada consulta: data, prompt enviado, síntese da resposta efetiva, decisão do estudante, arquivo/commit eventualmente modificado e captura ou link da evidência. Não se apresentam respostas fictícias como se fossem produzidas na conta do aluno.

B.2 – Prompts de planejamento e implantação

Os seis primeiros prompts orientam a definição do projeto, a segurança e o uso dos serviços externos. O estudante deve conferir cada resposta com a documentação técnica antes de executar qualquer configuração.

Prompt 1 – Contexto acadêmico

Sou aluno do primeiro semestre de Engenharia de Software e estou desenvolvendo o PortfolioHUB como entrega final de Bootcamp I. O site estático apresenta meus projetos públicos e utiliza GitHub Pages, HTML, CSS e JavaScript. Atue como assistente técnico e proponha perguntas de diagnóstico antes de recomendar alterações.

Finalidade: Avaliar se a IA compreendeu escopo, público e limitações.

Prompt 2 – Planejamento da implantação

Com base nas seis frentes planejamento, configuração GitHub, usuários e segurança, controle de acesso, testes e apresentação, crie um cronograma operacional com dependências, entregáveis, riscos, responsável e critérios de aceite. Não afirme que ações foram executadas.

Finalidade: Comparar etapas sugeridas à matriz de requisitos.

Prompt 3 – Arquitetura e GitHub

Revise esta arquitetura: frontend em docs/index.html, docs/style.css e docs/app.js, consulta somente à API pública do GitHub, sem credenciais no navegador. Identifique riscos de disponibilidade, performance, links externos e manutenção.

Finalidade: Verificar coerência de decisões arquiteturais.

Prompt 4 – Segurança e dados pessoais

Revise uma política para site estático hospedado no GitHub Pages. Considere menor privilégio, autenticação multifator da conta, dados públicos, segredos, permissões de colaboradores, LGPD e resposta a incidentes. Diferencie controles técnicos existentes de recomendações.

Finalidade: Priorizar mitigação de exposição acidental.

Prompt 5 – Colaboração

Descreva boas práticas para branch, commits pequenos, pull requests, issues e revisão de código de um projeto acadêmico individual que poderá receber contribuições. Inclua comandos Git e critérios de aprovação.

Finalidade: Comparar com o guia de colaboração.

Prompt 6 – Google Workspace

Proponha como organizar Docs, Drive e Agenda para documentar o PortfolioHUB, sem afirmar existência de integrações privadas. Explique diferenças entre hiperlinks, incorporação autorizada, OAuth e APIs.

Finalidade: Evitar alegações de integração não implementada.

B.3 – Prompts de testes e aperfeiçoamento

Os prompts seguintes destinam-se aos testes, à publicação e à melhoria do trabalho. As respostas devem ser registradas apenas após utilização real do Gemini.

Prompt 7 – Testes funcionais

Produza cenários manuais de teste de busca, ordenação, erro de API, estado vazio, alternância de tema, links externos e compatibilidade móvel. Para cada um, forneça pré-condição, passos e resultado esperado, mas não invente resultados obtidos.

Finalidade: Refinar o checklist de validação.

Prompt 8 – Acessibilidade

Revise práticas de HTML semântico, foco visível, navegação por teclado, nomes de controles, contraste e mensagens de erro para um site de portfólio simples. Sugira melhorias com prioridade e justificativa.

Finalidade: Apoiar revisão de experiência do usuário.

Prompt 9 – Implantação GitHub Pages

Crie um checklist de publicação em GitHub Pages a partir da pasta docs/ de uma branch principal. Inclua verificação da URL, assets, permissões e como documentar evidências reais.

Finalidade: Confrontar orientação com painel GitHub.

Prompt 10 – Revisão crítica

Assuma o papel de avaliador da entrega. Pergunte quais evidências existem para cada uma das seis etapas e indique afirmações que devem ser corrigidas caso não haja prova.

Finalidade: Identificar lacunas de demonstração.

Prompt 11 – Apresentação de 5 minutos

Elabore um roteiro de apresentação de até cinco minutos sobre o PortfolioHUB de um estudante iniciante, cobrindo planejamento, interface, Git, segurança, testes, IA e limitações. Sugira cenas e tempo por etapa.

Finalidade: Ensaiar comunicação clara e objetiva.

Prompt 12 – Melhorias futuras

Sugira um roadmap de evolução do PortfolioHUB em três níveis: manutenção imediata, funcionalidades intermediárias e expansão para backend autenticado. Separe propostas de funcionalidades já existentes.

Finalidade: Elaborar conclusão e perspectiva profissional.

REFERÊNCIAS

BRASIL. Lei nº 13.709, de 14 de agosto de 2018. Lei Geral de Proteção de Dados Pessoais (LGPD).

CENTRO UNIVERSITÁRIO DE BRASÍLIA. Desafio – Entrega Final: Implantação PortfolioHUB + IA Gemini. Material da disciplina Bootcamp I, 2026.

GITHUB. GitHub Docs: repositórios, GitHub Pages, controle de versão, pull requests e GitHub Actions. Documentação técnica institucional.

GOOGLE. Google Gemini e Google Workspace: documentação dos serviços e práticas de compartilhamento. Documentação institucional.

ISO/IEC. ISO/IEC 27001: Information security management systems. Referência conceitual para segurança da informação.

MORAIS, Juliano. Bootcamp I – Turma B – Entrega Final. Trabalho acadêmico fornecido como referência de organização documental, 2024.

NATIONAL INSTITUTE OF STANDARDS AND TECHNOLOGY. Cybersecurity Framework 2.0. Referência conceitual para gestão de riscos.